

LA PROGRAMMATION EN LANGAGE C

CHAPITRE 2 : LES VARIABLES

MIP/ S3

Pr S.ELFILALI

sanaa.elfilali@etu.univh2c.ma

Une variable
les variables typées
Les caractéristiques d'une variable
Les types de variable
Les types numériques entiers
Les types rationnels
Déclaration d'une variable
Initialisation
Déclaration d'une constante
Identificateurs
Incrémentatation / décrementation
Préfixe et postfixe
Conversion des types

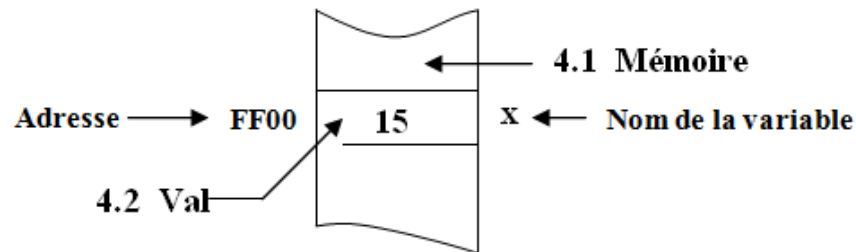
UNE VARIABLE ?

Une variable est un objet

Repéré par son
nom

Pouvant
contenir des
données

Pourront être
modifiées lors de
l'exécution du
programme.



LES VARIABLES TYPÉES

Les variables en langage C sont typées,

les données possèdent un type

elles sont stockées dans la mémoire

occupent un nombre d'octets

dépendant du type de donnée stockée.

LES CARACTÉRISTIQUES D'UNE VARIABLE

Nom

- Unique pour chaque variable
- Commence toujours par une lettre
- Différenciation minuscule-majuscule

type

- Conditionne le format de la variable en mémoire
- Peut être soit un type standard ou un type utilisateur

Valeur

- Peut évoluer pendant l'exécution
- initialisation grâce à l'opérateur d'affectation

Adresse

LES TYPES DE DONNÉES

Types de variable

Char	→	caractères
Int	→	entiers
short [int]	→	entiers courts
long [int]	→	entiers longs
Float	→	nombres décimaux
Double	→	nombres décimaux de précision supérieure
long double	→	nombres décimaux encore plus précis
unsigned int	→	entier non signé

TYPES NUMÉRIQUES ENTIERS

Type de donnée	Taille (en octets)	Plage de valeurs acceptée
char	1	-128 à 127
unsigned char	1	0 à 255
short int	2	-32768 à 32767
unsigned short int	2	0 à 65535
int	2	-32768 à 32767
unsigned int	2	0 à 65535
long int	4	-2 147 483 648 à 2 147 483 647

LES TYPES RATIONNELS

Type de donnée	Taille (en octets)	Plage de valeurs acceptée
float	4	-3.4×10^{-38} à 3.4×10^{38}
double	8	1.7×10^{-308} à 1.7×10^{308}
long double	10	3.4×10^{-4932} à 3.4×10^{4932}

DÉCLARATION D'UNE VARIABLE

Les déclarations permettent de :

réserver de l'espace en mémoire
pour les variables.

```
Type  nom_de_la_variable [= valeur] ;
```

DÉCLARATION D'UNE VARIABLE

Type nom_de_la_variable [= valeur] ;

Exemple:

int nb;

float pi = 3.14;

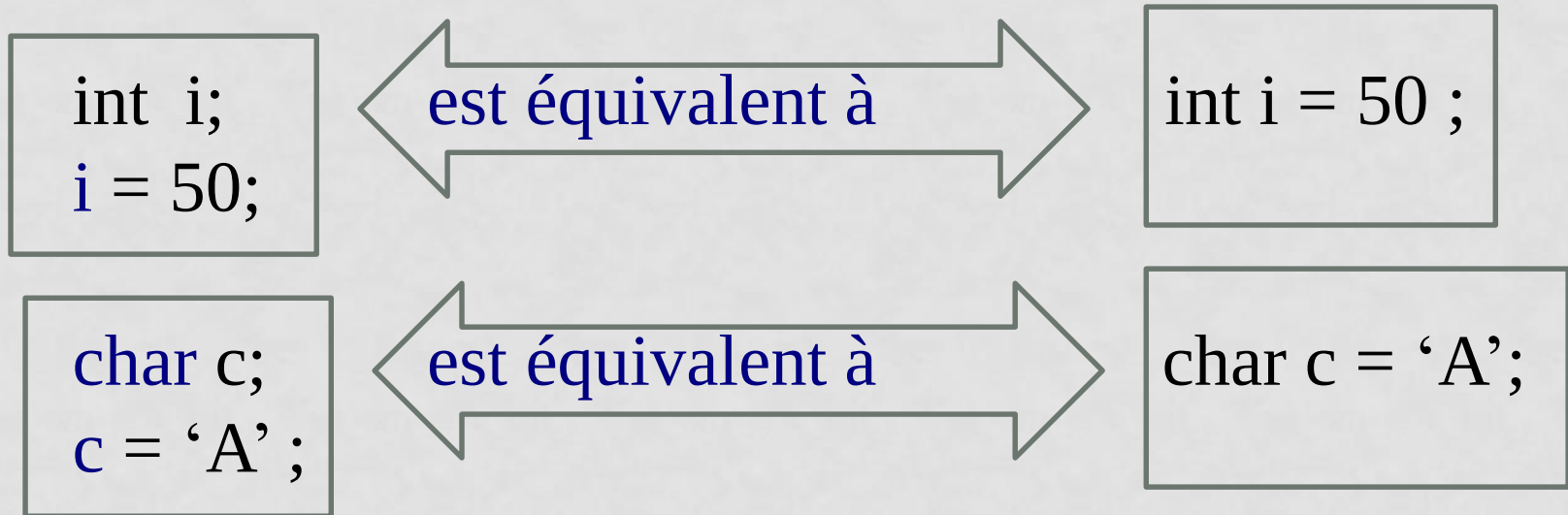
char c = 'a';

long i,j,k;

double r = 6.2879821365;

LES INITIALISATIONS

C permet l'initialisation des variables dans la zone de déclaration



LES DECLARATIONS DE CONSTANTES

1ere méthode :

définition d'un symbole à l'aide de la **directive** de compilation **#define**.



Le compilateur ne réserve pas de place en mémoire

Syntaxe : **#define** **identificateur** texte(valeur)

Exemple:

```
#define      PI  3.14159
void main()
{
    float perimetre, rayon = 8.7;
    perimetre = 2*rayon*PI;
    ....
}
```

LES DECLARATIONS DE CONSTANTES

2ème méthode :

déclaration d'une variable, dont la valeur sera constante pour tout le programme.

➤ **Le compilateur réserve de la place en mémoire**

Exemple

```
void main()
{
    const float PI = 3.14159;
    const int JOURS = 5;
    float perimetre, rayon = 8.7;
    perimetre = 2*rayon*PI;
    ....
    JOURS = 3;
    ....
}
```

Le compilateur réserve de la place en mémoire (ici 4 octets).

*/*ERREUR !*/ On ne peut changer la valeur d'une const.*

IDENTIFICATEURS

Les identificateurs nomment les objets C (fonctions, variables ...)

- **C'est une suite de lettres ou de chiffres.**
- **Le premier caractère est obligatoirement une lettre.**
- **Le caractère _ (souligné) est considéré comme une lettre.**

IDENTIFICATEURS

Le C distingue les minuscules des majuscules.

Exemples :

abc, Abc, ABC sont des identificateurs valides et tous différents.

Identificateurs valides :

xx	z2	somme_5	_position
Noms	surface	fin_de_fichier	VECTEUR

Identificateurs invalides :

5eme
x#z
no-commande
taux change

commence par un chiffre
caractère non autorisé (#)
caractère non autorisé (-)
caractère non autorisé (espace)

IDENTIFICATEURS

Un identificateur ne peut pas être un mot réservé du langage :

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

RQ : Les **mots réservés** du langage C doivent être écrits **en minuscules.**

INCRÉMENT ET DÉCREMENT

- C a deux opérateurs spéciaux pour incrémenter (ajouter 1) et décrémenter (retirer 1) des variables entières
 - ++ **increment** : `i++` ou `++i` est équivalent à `i += 1` ou `i = i + 1`
 - -- **decrement**
- Ces opérateurs peuvent être :
 - préfixés (avant la variable)
 - postfixés (après)

“i” vaudra 6

“j” vaudra 3

“i” vaudra 7

```
int i = 5, j = 4;  
i++;  
--j;  
++i;
```

PRÉFIXE ET POSTFIXE

```
#include <stdio.h>
```

```
int main(void)  
{
```

```
    int    i, j = 5;
```

```
    i = ++j;  
    printf("i=%d, j=%d\n", i, j);
```

```
    j = 5;  
    i = j++;
```

```
    printf("i=%d, j=%d\n", i, j);
```

```
    return 0;
```

```
}
```

équivalent à:

1. j++;
2. i = j;

équivalent à:

1. i = j;
2. j++;

i=6, j=6
i=5, j=6

LES CONVERSIONS DE TYPES

Le langage C permet d'effectuer des opérations de conversion de type. On utilise pour cela l'opérateur de "cast" ().

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int i=0x1234, j;
    char d,e;
    float r=89.67,s;
    j = (int)r;
    s = (float)i;
    d = (char)i;
    e = (char)r;
    printf("Conversion float -> int: %5.2f -> %d\n",r,j);
    printf("Conversion int -> float: %d -> %5.2f\n",i,s);
    printf("Conversion int -> char: %x -> %x\n",i,d);
    printf("Conversion float -> char: %5.2f -> %d\n",r,e);
    printf("Pour sortir frapper une touche ");
    getch();    // pas getchar
}
```

Conversion float -> int: 89.67 -> 89
Conversion int -> float: 4660 -> 4660.00
Conversion int -> char: 1234 -> 34
Conversion float -> char: 89.67 -> 89
Pour sortir frapper une touche

AFFICHAGE

La fonction printf()

- Sortie de nombres ou de texte à l'écran
- Printf une fonction de la bibliothèque `stdio.h`

AFFICHAGE D UN TEXTE

```
printf("BONJOUR");
```

pas de retour à la ligne du curseur après l'affichage,

```
printf("BONJOUR\n");
```

affichage du texte, puis retour à la ligne du curseur.

AFFICHAGE D'UNE VARIABLE

Fonction de la bibliothèque stdio.h

```
printf("%format",variable);
```

format :

d

- pour les décimaux,

x

- pour les hexadécimaux,

c

- pour un caractère, etc.

AFFICHAGE D'UNE VARIABLE

Fonction de la bibliothèque stdio.h

Format : `printf("%format",variable);`

Exemples :

```
printf("bonjour");
```

```
printf("la valeur est : %d ",i);
```

```
printf("les valeurs sont : %d et %d",i,j);
```

SPÉCIFICATEURS DE FORMAT POUR PRINTF()

SYMBOLE	TYPE	IMPRESSION COMME
%d ou %i	int	Entier relatif
%u	int	Entier naturel (unsigned)
%o	int	Entier exprimé en octal
%x	Int	Entier exprimé en hexadécimal
%c	int	caractère
%f	double	rationnel en notation décimale
%e	double	rationnel en notation scientifique
%s	char*	chaîne de caractères

AFFICHAGE MULTIPLE

```
printf("format1.... formatN",variable1,....,variableN);
```

A suivre!!

Exemple printf

Exemple 1:

```
Int q=100;  
Float p = 60.50;  
Char t = 'A';  
printf("quantite = %d \n ",q);  
printf(" Prix= %f\n ",p);  
printf(" Taille= %c \n ",t);  
printf(" Prix Total =%f \n ",q*p);
```

ou bien

```
printf("Quantite = %d\nPrix = %f\nTaille = %c\nPrix total = %f",q,p,t,q*p);
```

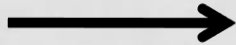
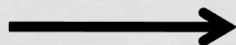
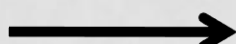
Imprime :

```
Quantite = 100  
Prix = 60.500000  
Taille = A  
Prix total = 6050,000000
```

Exemple printf

Exemple 2:

float val = 25.1234;

<code>printf("%f",val);</code>		<i>/* affiche : 25.123400 */</i>
<code>printf("%.0f",val);</code>		<i>/* affiche : 25 */</i>
<code>printf("%.1f",val);</code>		<i>/* affiche : 25.1 */</i>
<code>printf("%.2f",val);</code>		<i>/* affiche : 25.12 */</i>
<code>printf("%e",val);</code>		<i>/* affiche : 2.512340e+01 */</i>
<code>printf("%.3e",val);</code>		<i>/* affiche : 2.512e+01 */</i>

EXERCICE1.2

- ❑ Ecrire un programme C permettant de créer 2 variables
2 caractères initialisés à 65 et 'A'
- ❑ Afficher ces variables sous forme d'entiers et de caractères

SOLUTION EXERCICE 1.2

```
#include <stdio.h>
#include <conio.h>

void main()
{
    char u,v;
    u=65;
    v='A';
    printf("u = %d, %c",u,u);
    printf("v= %d, %c",v,v);
    getch();
}
```

LES OPÉRATEURS

Il existe 4 types d'opérateurs:

Arithmétiques

Logiques

Relationnels

Assignment.

OPÉRATEURS ARITHMÉTIQUES

+

- addition

-

- soustraction

*

- multiplication

/

- division (entière et rationnelle!)

%

- modulo (reste d'une div. entière)

EXERCICE 1.3

Ecrire un programme C déclarant 2 variables entières et affichant leur somme multipliée par 2.

SOLUTION EXERCICE 1.3

```
#include <stdio.h>
#include <conio.h>
void main ()
{
    int a,b,somme;
    a=10;
    b=50;
    somme=(a+b)*2;
    printf("résultat : %d\n",somme);
    printf("Appuyer sur une touche\n");
    getch();
}
```

OPÉRATEURS DE COMPARAISON

Ces opérateurs produisent une valeur booléenne vraie (**true**) ou fausse (**false**).

==

- égal à

!=

- différent de

<, <=, >, >=

- plus petit que, ...

OPÉRATEURS LOGIQUE

&& et logique (**and**)

|| ou logique (**or**)

! négation logique (**not**)

OPÉRATEURS LOGIQUE

Les résultats des opérations de comparaison et des opérateurs logiques sont du type int:

- la valeur **1** correspond à la valeur booléenne vrai
- la valeur **0** correspond à la valeur booléenne faux

OPÉRATEURS LOGIQUE

Les opérateurs logiques considèrent :

toute valeur **différente de zéro** comme vrai

Et toute **valeur zéro** comme faux.

OPÉRATEURS DE MANIPULATION DE BITS

Fonction	Symbole	Remarques
ET	&	$c = a \& b$; $\rightarrow$ si $a = 1001\ 0011$ et $b = 0101\ 0110$, $c = 0001\ 0010$
OU		$c = a b$; $\rightarrow$ si $a = 1001\ 0011$ et $b = 0101\ 0110$, $c = 1101\ 0111$
OU exclusif	^	$c = a \wedge b$; $\rightarrow$ si $a = 1001\ 0011$ et $b = 0101\ 0110$, $c = 1100\ 0101$
Décalage à droite	>>	$(a >> 2)$ décale la valeur de a de 2 bits vers la droite
Décalage à gauche	<<	$(a << 2)$ décale la valeur de a de 2 bits vers la gauche
Complément à un	~	$c = \sim a \rightarrow$ si $a = 1001\ 0011$, $c = 0110\ 1100$

```
unsigned char u_c = 1;
```

0	0	0	0	0	0	0	1
---	---	---	---	---	---	---	---

```
u_c = u_c << 1;
```

0	0	0	0	0	0	1	0
---	---	---	---	---	---	---	---

```
u_c = u_c << 2;
```

0	0	0	0	1	0	0	0
---	---	---	---	---	---	---	---

```
u_c = u_c >> 3;
```

0	0	0	0	0	0	0	1
---	---	---	---	---	---	---	---

LES OPÉRATEURS D'AFFECTATION

En pratique, nous retrouvons souvent des affectations comme:

$i = i + 2$

En C, nous utiliserons plutôt la formulation plus compacte:

$i += 2$

L'opérateur $+=$ est un *opérateur d'affectation*.

LES OPÉRATEURS D'AFFECTATION

Somme = somme + i;

Ici, la valeur de la variable **somme** est additionnée à **i** puis réassignée (ensuite !) à la même variable **somme**.

LES OPÉRATEURS D'AFFECTATION

Pour la plupart des expressions de la forme:

$$\text{expr1} = (\text{expr1}) \text{ op } (\text{expr2})$$

$$\text{expr1} \text{ op} = \text{expr2}$$

+=

ajouter à

-=

diminuer de

***=**

multiplier par

/=

diviser par

%=

modulo

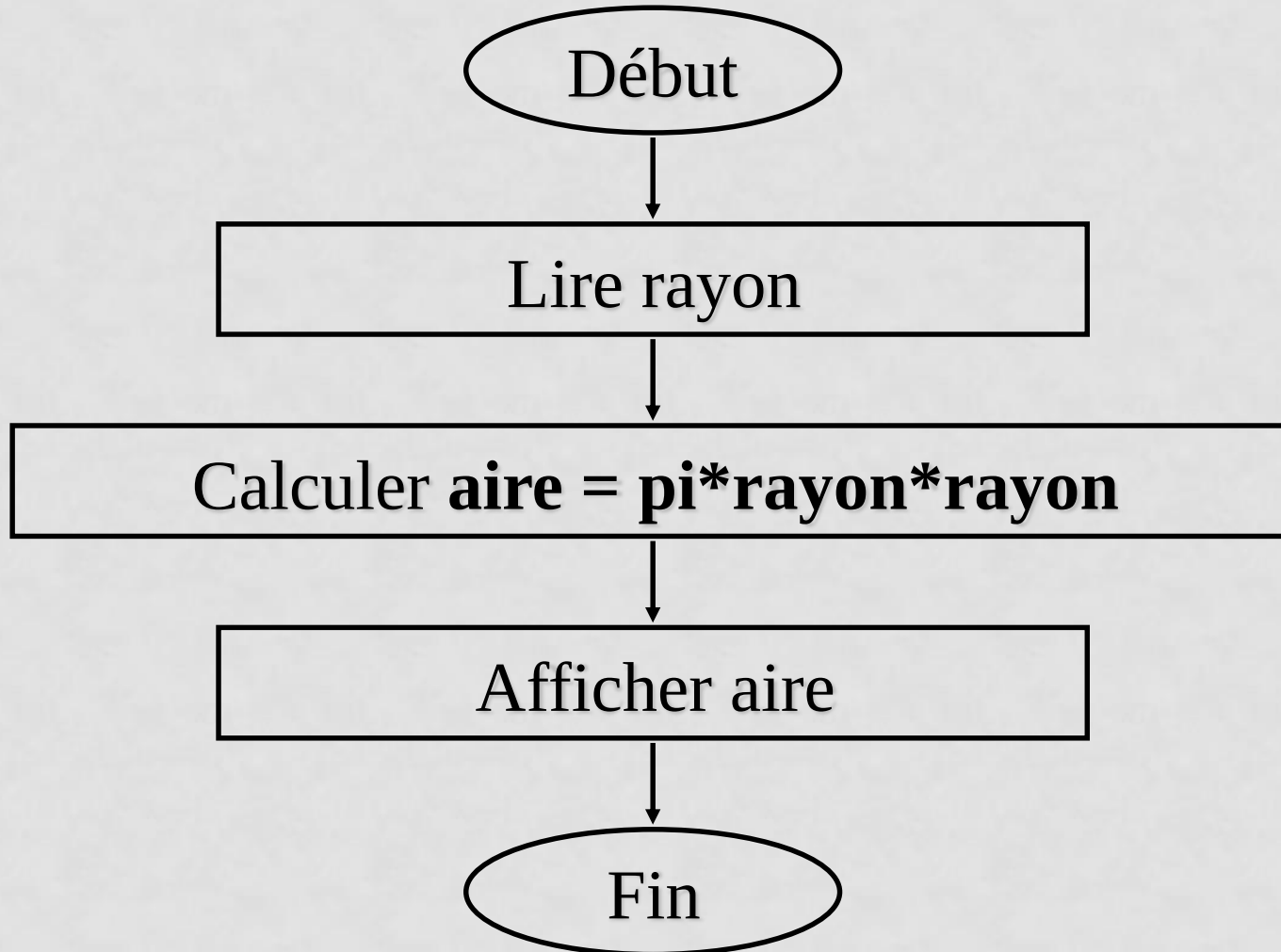
LES OPÉRATEURS ABRÉGÉS (OPÉRATEURS D'ASSIGNEMENT)

opérateur abrégé	Expression abrégée	Expression équivalente
<code>+=</code>	<code>c += 3;</code>	<code>c = c + 3;</code>
<code>-=</code>	<code>c -= 3;</code>	<code>c = c - 3;</code>
<code>*=</code>	<code>c *= 3;</code>	<code>c = c * 3;</code>
<code>/=</code>	<code>c /= 3;</code>	<code>c = c / 3;</code>
<code>%=</code>	<code>c %= 3;</code>	<code>c = c % 3;</code>

Priorité des opérateurs

N priorité	Type d'opérateur	Opérateurs	Associativité
15	Primaire	() [] . ->	g à d
14	Unaire	++ -- (nom Type) * & sizeof + - ! ~	g à d
13	Arithmétique	* / %	g à d
12	Arithmétique	+ -	g à d
11	De manipulation de bits	>> <<	g à d
10	De relation	> < >= <=	g à d
9	De relation	== !=	g à d
8	De manipulation de bits	&	g à d
7	De manipulation de bits	^	g à d
6	De manipulation de bits		g à d
5	Logique	&&	g à d
4	Logique		g à d
3	Conditionnel	?:	g à d
2	D'affectation	= += -= *= /= = ^=	g à d

ALGORITHME AIRE



PROGRAMME AIRE

```
#include <stdio.h>
void main()
{
    float aire, pi, rayon;
    rayon = 3.7;
    pi = 3.14159;
    aire = pi*rayon*rayon;
    printf("Aire est %f\n", aire);
}
```

LA FONCTION SCANF()

Définition :

La fonction scanf() :

- appartenant à la bibliothèque **stdio.h**
- permet de lire des données au clavier et de les affecter à des variables.
- scanf (scan : formatted) peut être utilisée pour la saisie des données formatées (lettre, chiffre ou chaîne de caractère)
C'est une fonction variadique (peut prendre un nombre variable de paramètres)
La fonction retourne le nombre de variable affectées par la saisie
=> Permet de vérifier si la saisie s'est bien passée

LA FONCTION SCANF()

Syntaxe :

```
scanf("format_variable1 format_variable 2 ....format_variable n", &variable_1,  
      &variable_2, ..., & variable_n);
```


LA FONCTION SCANF() EXEMPLE

```
char alpha;
```

```
int i;
```

```
float r;
```

```
scanf("%c",&alpha);  $\Longrightarrow$  /* saisie d'un caractère */
```

```
scanf("%d",&i);  $\Longrightarrow$  /* saisie d'un nombre entier */
```

```
scanf("%x",&i);  $\Longrightarrow$  /* saisie d'un nombre entier en hexadécimal*/
```

```
scanf("%f",&r);  $\Longrightarrow$  /* saisie d'un nombre réel */
```

LA FONCTION SCANF() EXEMPLE

```
#include <stdio.h>
#include <string.h>
void main()
{
    // declaration des variables
    int V1,V2,V3;
    char S1[30], S2[30];
    int Somme ;
    // lecture des données
    printf("votre nom : "); scanf("%s", S1);
    printf("votre prenom : ") ; scanf("%s", S2);

    // lecture et traitement
    printf("la saisie des donnees :\n ");
    Somme=scanf("%d%d%d",&V1,&V2,&V3);
    //affichage

    printf("%s %s vous avez saisi %d nombres ",S1, S2, Somme);

}
```

```
votre nom : ELFILALI
votre prenom : SANAA
la saisie des donnees :
3
4
5
ELFILALI SANAA vous avez saisi 3 nombres
```

AIRE D UN CERCLE (AMÉLIORATION)

```
/* Premier programme en C */  
#include <stdio.h>  
void main()  
{  
    float aire, pi, rayon;  
    // rayon = 3.7;  
    printf("Entrez une valeur pour rayon : ");  
    scanf("%f", &rayon);  
    pi = 3.14159;  
    aire = pi*rayon*rayon;  
    printf("Aire est %f\n", aire);  
}
```

AIRE D'UN CERCLE (AMÉLIORATION)

```
printf("Entrez une valeur pour rayon : ");
```

```
// Ce printf n'a pas de \n (nouvelle ligne)
```

```
scanf("%f", &rayon);
```

```
//scanf prend la valeur réelle entrée (%f) et l'assigne à la  
variable rayon.
```

AIRE D'UN CERCLE (AMÉLIORATION) EXÉCUTION DU PROGRAMME

Entrez une valeur pour rayon : 3.7

Aire est 43.008369

La fonction Scanf() Exemple

Saisie de plusieurs variables.

```
{
```

```
float a, b, c, det ;
```

```
printf("Donnez les valeurs de a,b et c : ");
```

```
/* Saisie de a, b, c */
```

```
scanf("%f %f %f",&a,&b,&c);
```

```
/* Calcul du déterminant */
```

```
det = b*b- 4*a*c;
```

```
printf("Le déterminant = %.2f", det);
```

```
}
```

La fonction Scanf() Exemple

Tampon de scanf

Les informations tapées au clavier sont d'abord mémorisées dans un emplacement mémoire appelé *tampon* (**buffer**).

elles sont mises à la disposition de **scanf** après l'activation de la touche <Entrée>

➤ Pour les nombres :

scanf avance jusqu'au premier caractère différent d'un séparateur (espace, tabulation, retour à la ligne)

puis **scanf** prend en compte tous les caractères jusqu'à la rencontre d'un séparateur ou d'un caractère invalide ou en fonction du largeur.

➤ Pour les caractères %c :

- **scanf** prend le caractère courant qu'il soit séparateur ou non séparateur.

La fonction Scanf() Exemple

Tampon de scanf : résoudre le problème de la saisie des caractères

- Pour les caractères %c : il faut laisser un espace avant le % : exemple
`scanf(" %c",&ch);`

Un espace



Convertir le caractère
majuscule en minuscule
Et
Minuscule en majuscule

Exercice :

écrire un programme qui permet de convertir un caractère
saisi au clavier en majuscule et minuscule

Convertir le caractère majuscule en minuscule Et Minuscule en majuscule

```
#include <stdio.h>
void main(void)
{
    char ch;

    printf("veuillez saisir un caractere : "); scanf(" %c",&ch);

    printf("caractere saisi est : %c\t \n
           en maj = %c \t en min = %c\n", ch,toupper(ch), tolower(ch));
}
```

```
veuillez saisir un caractere : S
caractere saisi est : S
    en maj = S        en min = s
```

Convertir le caractère majuscule en minuscule Et Minuscule en majuscule

```
#include <stdio.h>
#include <ctype.h>
#include <time.h>
void main(void)
{

    char ch;
    time_t    t = time(NULL);
    printf(" \t exemple cours : Exercice conversion      %s\n", asctime(localtime(&t)));
    printf(" \t ===== \n\n");
    printf("*****\n");
    printf("Realise par : SMI S3 \n");
    printf("Encadre par Professeur: S.ELFILAL\n");
    printf("*****\n");
    printf("veuillez saisir un caractere : ");    scanf(" %c",&ch);

    printf("caractere saisi : %c\t maj toupper= %c\n", ch,toupper(ch));
    printf("caractere saisi : %c\t min tolower= %c\n", ch, tolower(ch));
    putchar('\n');

}
```

La moyenne de deux nombres entrés au clavier

Exercice :

écrire un programme qui permet de calculer la moyenne de deux nombres réels entrés au clavier et l'afficher :

La moyenne de deux nombres entrés au clavier

Exercice : écrire un programme qui permet de calculer la moyenne de deux nombres réels entrés au clavier et l'afficher :

```
#include <stdio.h>
void main()    // Point d'entrée du programme
{
    /*Début du programme
    */
    float x, y;
    printf("\t\t\tCalcul de la moyenne\n\n");           /*Affiche le titre*/
    printf("Entrez le premier nombre : ");
    scanf("%f", &x);                                     /*Lecture du premier nombre*/
    printf("Entrez le deuxième nombre : ");
    scanf("%f", &y);                                     /*Lecture du deuxième nombre*/
    printf("La valeur moyenne de %.2f et de %.2f est %.2f\n",x, y,(x+y)/2);
}    // Fin du programme
```